Neural Networks Cheatsheet

key equations, activations, optimizers, and architectures

Fundamentals

A feedforward network stacks affine transforms with nonlinearities. For layer l with input a^{l-1}:

$$z^l = W^l a^{l-1} + b^l, \qquad a^l = \sigma(z^l)$$

where $W^l \in \mathbb{R}^{n_l \times n_{l-1}}$, $b^l \in \mathbb{R}^{n_l}$, and σ is an elementwise activation.

Universal approximation: a single hidden layer with enough neurons can approximate any continuous function on a compact set (Cybenko 1989, Hornik 1991). Depth is preferred in practice for parameter efficiency.

Activation Functions

Name	$\sigma(x)$	$\sigma'(x)$
Sigmoid	$\frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x))$
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1-\tanh^2(x)$
ReLU	$\max(0,x)$	$\mathbf{1}[x>0]$
Leaky ReLU	$\max(\alpha x, x)$	1 or α
GELU	$x\,\Phi(x)$	(smooth ReLU)
SiLU/Swish	$x\,\sigma(x)$	(smooth, non-mono.)

Softmax (output for K-class classification):

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad \sum_i p_i = 1$$

Loss Functions

Regression

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Binary classification

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_i \left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

Multiclass (one-hot y)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_i \sum_k y_{ik} \log \hat{y}_{ik}$$

For softmax + CE together: $\nabla_z \mathcal{L} = \hat{y} - y$ (clean gradient).

Backpropagation

The four fundamental equations (Nielsen notation):

$$\begin{aligned}
\delta^L &= \nabla_a \mathcal{L} \odot \sigma'(z^L) \\
\delta^l &= \left((W^{l+1})^\top \delta^{l+1}\right) \odot \sigma'(z^l) \\
\frac{\partial \mathcal{L}}{\partial W^l} &= \delta^l (a^{l-1})^\top \\
\frac{\partial \mathcal{L}}{\partial b^l} &= \delta^l
\end{aligned}$$

Here $\odot$ is Hadamard product. Backprop = reverse-mode automatic differentiation applied to a DAG of tensor ops.

Optimizers

Let $g_t = \nabla \mathcal{L}(\theta_t)$, learning rate η.

SGD

$$\theta_{t+1} = \theta_t - \eta\, g_t$$

SGD + Momentum

$$v_{t+1} = \beta v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_{t+1}$$

RMSProp

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \quad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t}+\varepsilon}$$

Adam

$$\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \\
\theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}
\end{aligned}$$

Defaults: β_1=0.9, β_2=0.999, ε=10^{-8}.

Regularization

- **L2 (weight decay):** add $\lambda \|W\|_2^2$ to loss $\Rightarrow$ shrinks weights toward 0.
- **L1:** add $\lambda \|W\|_1 \Rightarrow$ induces sparsity.
- **Dropout:** zero each unit with prob. p at train time; scale by $1/(1-p)$.
- **Batch Norm:** normalize per mini-batch, then learnable γ, β: $\hat{x} = \frac{x-\mu_B}{\sqrt{\sigma_B^2+\varepsilon}}$, $y = \gamma\hat{x} + \beta$.
- **Layer Norm:** normalize across features (used in Transformers).
- **Early stopping, data augmentation, label smoothing.**

Weight Initialization

Keeping activation variance stable across layers prevents vanishing/-exploding signals.

Xavier/Glorot (tanh, sigmoid):

$$\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

He/Kaiming (ReLU):

$$\text{Var}(W) = \frac{2}{n_{\text{in}}}$$

Architectures

CNN

Convolution output size with input W, filter F, padding P, stride S:

$$W_{\text{out}} = \left\lfloor \frac{W-F+2P}{S} \right\rfloor + 1$$

Params per conv layer: $F \cdot F \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$.

RNN / LSTM

Vanilla RNN: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$. LSTM adds input/forget/output gates to combat vanishing gradients over long sequences.

Transformer — Scaled Dot-Product Attention

$$\text{Attn}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Multi-head: h parallel attentions on projected Q, K, V, outputs concatenated and linearly projected. Complexity $\mathcal{O}(n^2 d)$ in sequence length n.

Training Tips

- Normalize inputs (zero mean, unit variance).
- Start with Adam at η=10^{-3}; fine-tune with SGD+mom.
- Use LR warmup + cosine/step decay for large models.
- Clip gradients ($\|g\|_2 \le c$) for RNNs/Transformers.
- Monitor train vs. val loss to detect over/underfitting.
- Prefer larger batches with linear LR scaling ($\eta \propto B$).
- Mixed-precision (fp16/bf16) $\approx 2\times$ speedup on modern GPUs.